

21 — Level 1 rejects: the auxiliary engines would run above 90 %

Proves · Level 1 rejecting because the auxiliaries would have to run above the 90 % limit.

Mechanics this scenario turns on

- Validation runs **before** Level 1. A plant that cannot carry its load is rejected there, with a different message and a different code path from the Level 1 rejections.
- The shaft generator is filled before any auxiliary starts (the main engine is already turning), and its output is a load **on** that main engine — which is why the ME figure exceeds propulsion. SG capacity scales with the number of running MEs.

Panels described below · The 100 % / 90 % gap · The plant · The response

Anything not described here — the mechanics above name the step that produced it; [00-ORIENTATION](#) Part 6 has the full number-to-step index.

Trust · characterisation snapshot, generated from the code. It detects change; it does NOT prove correctness. Figures marked *pending reference verification* have never been checked against anything outside the application.

Read after · scenario 20.

The second uncovered rejection branch. This one is subtle because **validation and Level 1 disagree on purpose**.

The 100 % / 90 % gap

```
ValidationService : aux utilisation > 100 % → error
Level 1           : aux load      > 90 % → reject the combination
```

The 10-point gap is deliberate: Level 2 needs headroom to redistribute, so Level 1 must not hand it a combination already at the limit. A plant sitting between 90 % and 100 % therefore **passes every form check and then fails to calculate** — which is exactly what this scenario captures.

The plant

```
ME 2 × 5 000 · no SG · AE 2 × 500 · propulsion 4 000 · hotel 950
```

- Validation: ME 40 % ✓ · hotel 950 ≤ AE capacity 1 000 ✓ · aux utilisation 95 % ≤ 100 % ✓
- Level 1, aux = 2: 950 / 1 000 = **95 %** → above the ceiling → [AuxOverloaded](#)
- Level 1, aux = 1: 950 > 500 → hotel uncovered → structural

Nothing survives.

The response

400

No feasible engine configuration: the auxiliary engines would run above 90% load in Transit mode. Increase auxiliary engine capacity or count, or reduce the hotel/mission power.

The message names the ceiling explicitly (90 %), because a user looking at a form that accepted 95 % has no other way to learn where the real limit is.

Takeaway: the aux ceiling is a *modelling* constraint, not a physical one — and it is the one place where passing validation is not enough.

KSailCalc manual test scenarios — calculation walkthrough · regenerated 2026-08-07 · numbers verified against commit 559aef2 and unchanged by the backend/client refactors (18 golden snapshots byte-identical)